UR THE INNOVATIVE UNIVERSITY	Challeneg2	Ing4-CYB/CCV	S7

Course Outcomes (Syllabus) avec le pourcentage		
CO	**Exercice**	**%**
CO1: Understand the fundamentals of PP and PA	Ex1	30%
CO2: Apply knowledge and develop hands-on skills in developing applications	Ex2	30%
CO3: Evaluate parallel code efficiency for HPC tuning and optimization	Ex3	40%

Nom[1] :.....**Prénom**:.....

The objective of this challenge is to implement a parallel program using MPI to generate multiple independent RSA key pairs (as complement to Challenge1). The RSA key-generation function is provided as sequential C code. The main task is to distribute independent calls to this function across MPI processes and evaluate the performance of the parallel implementation.

Part I: Sequential RSA Key Generation (Provided Code)
The sequential implementation of RSA key generation is provided in the files `rsa.c` and `rsa.c`. It includes the following functions:

- `gcd()`: calculates the greatest common divisor.
- `extended_gcd()`: computes the Bézout coefficients.
- `modular_inverse()`: calculates the modular inverse.
- `generate_rsa_keys()`: generates and verifies an RSA public and private key pair.

[1] Tous documents autorisés

The function `generate_rsa_keys()` takes two prime numbers p and q as input and calculates the corresponding RSA parameters:

$$N = pq, \qquad \varphi(N) = (p-1)(q-1).$$

It then determines the public exponent e and the private exponent d, satisfying:

$$\gcd(e, \varphi(N)) = 1$$

and

$$ed \equiv 1 \pmod{\varphi(N)}.$$

The provided implementation must be reused without rewriting its mathematical algorithms.

Part II: Parallel RSA Key Generation Using MPI (CO1, CO2, 12 points)

The objective is to parallelize multiple independent calls to `generate_rsa_keys()` using MPI. Each call generates one RSA key pair from a given pair of prime numbers.

Consider the following input data:

Table 1: Prime pairs for independent RSA key generation

Task	p	q
1	101	113
2	107	127
3	131	137
4	139	149
5	151	163
6	167	173
7	179	191
8	193	197
9	199	211
10	223	227
11	229	233
12	239	241
13	251	257
14	263	269
15	271	277
16	281	283

Complete the following tasks:

1. Develop a complete MPI C program named `rsa_mpi.c`, reusing the provided `generate_rsa_keys()` function.

2. Distribute the required independent key-generation tasks among the available MPI processes. Each task must be executed by exactly one process.

3. Each process must generate the RSA keys for its assigned tasks and store the resulting parameters (N, e, d).

4. Collect the results at the master process or use another documented approach to make all results available for verification.

5. Verify that the generated keys satisfy:

   $$\gcd(e, \varphi(N)) = 1$$

   and

   $$ed \bmod \varphi(N) = 1.$$

6. Ensure that every task is executed exactly once and that the parallel implementation produces the same mathematical results as the sequential implementation.

7. Provide the complete C source code, including MPI initialization, workload distribution, result collection, correctness verification, and MPI finalization.

Part III: Performance Evaluation (PartIII, 8 points)
Evaluate the performance of the parallel implementation by varying the number of MPI processes.

1. Measure the execution time of the sequential implementation for the complete set of key-generation tasks.

2. Measure the execution time of the MPI implementation using different numbers of processes, for example:

   $$P \in \{1, 2, 4, 8\},$$

   depending on the available hardware.

3. Calculate the speedup:

   $$S(P) = \frac{T_1}{T_P},$$

   where T_1 is the sequential execution time and T_P is the parallel execution time using P processes.

4. Calculate the parallel efficiency:

   $$E(P) = \frac{T_1}{PT_P}.$$

5. Present the results in a table containing the number of processes, execution time, speedup, and parallel efficiency.

6. Analyze the results, considering the number of independent tasks, workload distribution, process initialization, and communication overhead.

7. Since the individual key-generation tasks are computationally small, repeat the workload when necessary to obtain measurable execution times.

Please note that this implementation process separates the RSA implementation from the MPI program:

- `rsa.c` implements the RSA functions.
- `rsa_mpi.c` contains the MPI `main()` function.
- `rsa.h` declares the RSA functions.

First, compile `rsa.c` using the C compiler:

```
cc -Wall -Wextra -c rsa.c -o rsa.o
```

This command compiles the RSA implementation into an object file named `rsa.o`, without linking an executable.
Then, compile `rsa_mpi.c` using the MPI compiler; This command generates the object file `rsa_mpi.o`.

```
mpicc -Wall -Wextra -c rsa_mpi.c -o rsa_mpi.o
```

Link the two object files to generate the executable; This command produces the executable file `rsa_mpi`.

```
mpicc rsa_mpi.o rsa.o -o rsa_mpi
```

Run the program using four MPI processes:

```
mpirun -np 4 ./rsa_mpi
```

The program distributes independent RSA key-generation tasks among the four processes. Each process executes its assigned tasks and displays the generated RSA keys.
To compare the execution time with different numbers of MPI processes, run the program with different values of `-np`, for example:

```
mpirun -np 1 ./rsa_mpi
mpirun -np 2 ./rsa_mpi
mpirun -np 4 ./rsa_mpi
mpirun -np 8 ./rsa_mpi
```